

Inventory Reservation System

Reviewer Guide: Setup, Run and Test

Candidate	[Your name]
Position	[Role you are applying for]
Submitted	[Date]
Repository / archive	[Link or file name]
Stack	.NET 9 · ASP.NET Core · EF Core 9 · MySQL 8 · JWT · Serilog · xUnit · Docker

This guide takes you from a fresh download to a running, tested app in about **15 minutes**. Follow sections 2 to 4 in order. Sections 7 onwards explain how the app works.

Section	What you will do	Time
2. Set up and start the app	Install Docker, create the settings file, start everything with one command	5 min
3. Try it in Swagger	Log in, create a product, reserve, confirm, cancel, see expiry and errors	5 min (+2 min wait)
4. Run the automated tests	Run all 248 tests, including the 500-request test	3 min
5–6. Stop, reset, troubleshoot	Only if you need them	—

1. What the app does

An API for reserving products. A user reserves some stock, and it is held for **2 minutes**. In that time they can **confirm** it (buy it) or **cancel** it. If they do neither, the reservation **expires** and the stock goes back on sale.

The main promise: **you can never sell more than you have**, even when hundreds of people try to buy the last item at the same moment, and even when several copies of the API are running.

Proof: 500 people, 1 item

Stock: 1
Requests: 500 at the same moment
Success: 1
Rejected: 499
Overselling: 0

This test runs against a real MySQL database every time the tests run, and it has never oversold. You can run it yourself in section 4.

2. Set up and start the app

You do **not** need to install MySQL or create a database. Docker starts MySQL, creates the database and tables, and creates the admin user for you.

2.1 What you need

Tool	Needed for	Check it with
Docker Desktop (running)	Starting the app	<code>docker --version</code>
.NET 9 SDK	Running the automated tests only	<code>dotnet --version</code> (shows 9.x)
Free ports 8080 and 3306	The API and MySQL	If one is taken, see step 2 below

2.2 Start it

1. Open a terminal in the project folder (the folder that contains `docker-compose.yml`).
2. Create your settings file by copying the example:

```
# macOS / Linux / Git Bash
cp .env.example .env

# Windows PowerShell
Copy-Item .env.example .env
```

- Open `.env` in any text editor and replace every `change-me` value with your own. Remember the admin password: you will use it to log in.

```
MYSQL_ROOT_PASSWORD=<choose a password>
MYSQL_PASSWORD=<choose another password>
JWT_SIGNING_KEY=<any random text, at least 32 characters>
DEMO_ADMIN_USERNAME=admin
DEMO_ADMIN_PASSWORD=<choose the admin password>
SWAGGER_ENABLED=true
```

If port 3306 or 8080 is already used on your computer, change `MYSQL_PORT` (for example to 3307) or `API_PORT` in the same file.

- Start the app:

```
docker compose up --build
```

- Wait about 1 minute the first time. In a **second** terminal, check that both parts are ready:

```
docker compose ps      # both lines should say (healthy)
```

- Open these links in your browser:

Link	What you should see
http://localhost:8080/health/ready	Healthy (the API can reach the database)
http://localhost:8080/swagger	The Swagger page listing all endpoints

That is it: the app is running. Continue with section 3 to try it.

3. Try it in Swagger (step by step)

Follow these steps in order to try every feature: log in, create a product, reserve, buy, cancel, watch a reservation expire, and see the main errors. It takes about **5 minutes**, plus a 2-minute wait for the expiry step. Your ids and times will differ from the screenshots.

Before you start: the app is running (section 2), and you know the admin username and password you put in `.env`.

How to use Swagger	
Expand an endpoint	Click its coloured row (e.g. the green POST bar).
Enable editing	Click Try it out (top right of the expanded endpoint).
Send the request	Edit the request body or the <code>id</code> field, then click Execute .
Read the result	Scroll to Server response : status code and response body.

Step 0. Open Swagger UI

Browse to <http://localhost:8080/swagger>. Protected endpoints show a padlock; login and health do not.

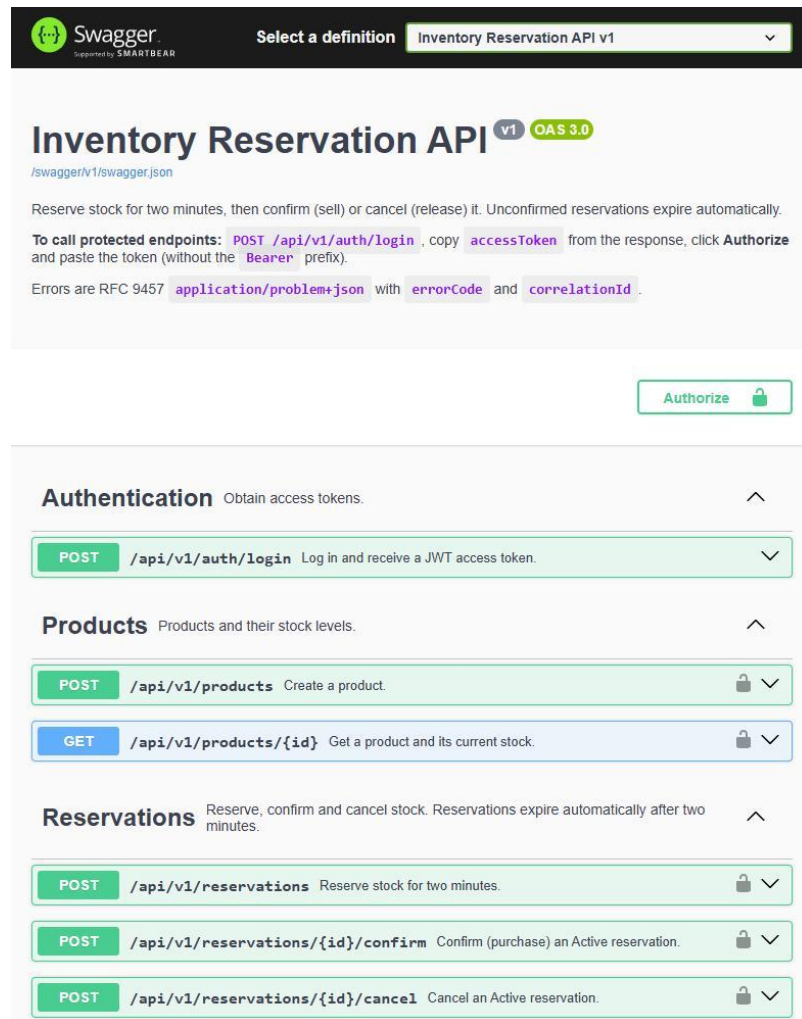


Figure 1: Swagger UI with the Authorize button and padlocked endpoints

Step 1. Log in and get a JWT

Endpoint: **POST /api/v1/auth/login**

- Expand the endpoint → **Try it out** → replace the body with your credentials → **Execute**.

```
{
  "username": "admin",
  "password": "<DEMO_ADMIN_PASSWORD from .env>"
}
```

Expected: **200 OK** the body contains **accessToken**, **tokenType**: "Bearer" and **expiresAtUtc** (60 minutes).

- Copy the value of accessToken** (without the quotes). You need it in Step 2.



Figure 2: Login response (token shortened in this image)

Step 2. Authorize Swagger with the token

- Scroll to the top and click **Authorize**.
- Paste the token into **Value**. Do not type the word Bearer; Swagger adds it.
- Click **Authorize**, then **Close**. The padlocks now appear closed.

Expected: **Authorized** the dialog shows "Authorized" and the value is masked.

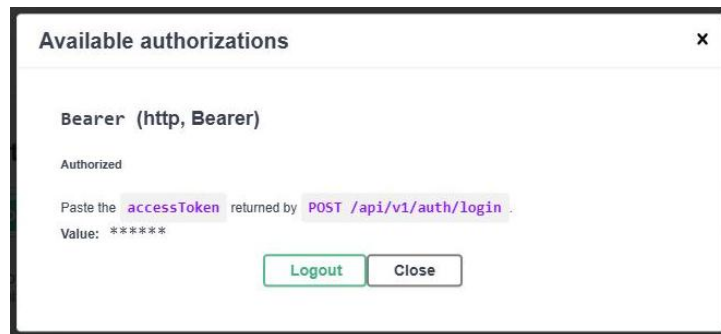


Figure 3: Authorize dialog after a successful authorization

Step 3. Create a product (Admin only)

Endpoint: POST /api/v1/products

```
{
  "sku": "DEMO-PHONE-001",
  "name": "Demo Phone",
  "totalQuantity": 3
}
```

Expected: **201 Created** reservedQuantity: 0, soldQuantity: 0, availableQuantity: 3. A Location header points to the new product.

- **Copy the product id** for the next steps. (Running it twice returns **409 DUPLICATE_SKU**; use another SKU.)



Figure 4: Product created

Step 4. Reserve stock

Endpoint: POST /api/v1/reservations

```
{  "productId": "<product id from Step 3>",  "quantity": 2}
```

Expected: **201 Created** status: "Active", and expiresAtUtc is 2 minutes after createdAtUtc.

- **Copy the reservationId.** Complete Step 6 within 2 minutes, otherwise the reservation expires.

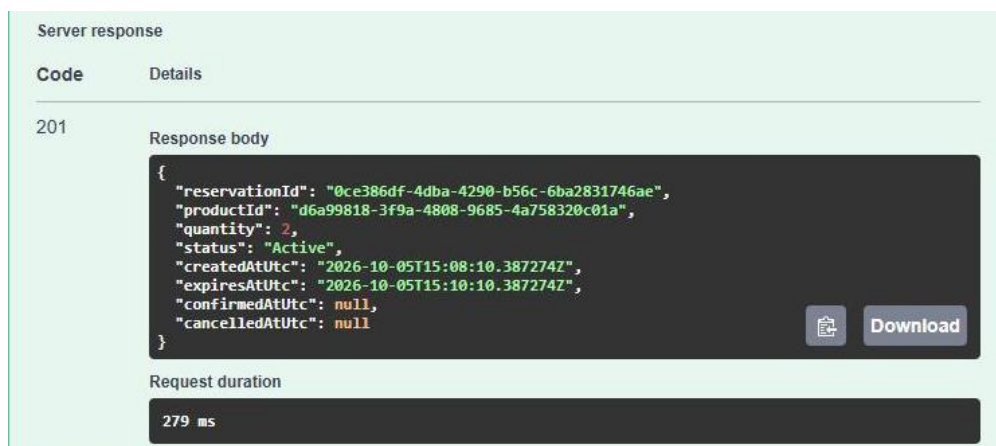


Figure 5: Active reservation holding 2 units

Step 5. Try to oversell (insufficient stock)

Endpoint: POST /api/v1/reservations (same body, quantity 2)

The product has 3 in stock and 2 are already reserved, so only 1 is left. Asking for 2 more must fail.

Expected: **409 Conflict** ProblemDetails with title: "Insufficient inventory" and errorCode: "INSUFFICIENT_STOCK". Nothing is reserved.

Server response

Code	Details
409	Error: Conflict Response body <pre>{ "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10", "title": "Insufficient inventory", "status": 409, "detail": "Product 'd6a99818-3f9a-4808-9685-4a758320c01a' does not have 2 unit(s) available.", "instance": "/api/v1/reservations", "errorCode": "INSUFFICIENT_STOCK", "correlationId": "60c3f877ee66443390cbf85d77630d2c" }</pre> Download

Figure 6: Overselling is rejected

Step 6. Confirm the reservation (purchase)

Endpoint: POST /api/v1/reservations/{id}/confirm

- Paste the **reservationId** from Step 4 into the **id** field → **Execute** (no body).

Expected: **200 OK** status: "Confirmed" with **confirmedAtUtc** set. The 2 reserved units are now sold.

Server response

Code	Details
200	Response body <pre>{ "reservationId": "0ce386df-4dba-4290-b56c-6ba2831746ae", "productId": "d6a99818-3f9a-4808-9685-4a758320c01a", "quantity": 2, "status": "Confirmed", "createdAtUtc": "2026-10-05T15:08:10.387274Z", "expiresAtUtc": "2026-10-05T15:10:10.387274Z", "confirmedAtUtc": "2026-10-05T15:09:39.443715Z", "cancelledAtUtc": null }</pre> Download Request duration 268 ms

Figure 7: Reservation confirmed

Step 7. Confirm the same reservation again

Endpoint: POST /api/v1/reservations/{id}/confirm (same id)

Expected: **409 Conflict** errorCode: "INVALID_RESERVATION_STATE" and the message "...is already Confirmed." The same item cannot be bought twice.

Server response

Code	Details
409	Error: Conflict Response body <pre>{ "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10", "title": "Invalid reservation state", "status": 409, "detail": "Reservation '2d7d3b56-2efb-4888-bf44-7ccee46317db' is already Confirmed.", "instance": "/api/v1/reservations/2d7d3b56-2efb-4888-bf44-7ccee46317db/confirm", "errorCode": "INVALID_RESERVATION_STATE", "correlationId": "05117de30a8349ea8ffac8f8906cf2c0" }</pre> Download

Figure 8: Second confirm is rejected

Step 8. Check the product stock

Endpoint: GET /api/v1/products/{id}

- Paste the product `id` from Step 3 → **Execute**.

Expected: **200 OK** `totalQuantity: 3, reservedQuantity: 0, soldQuantity: 2, availableQuantity: 1.`



Figure 9: Stock after the sale

Step 9. Reserve and cancel

- POST /api/v1/reservations with { "productId": "<product id>", "quantity": 1 } → 201. Copy the new `reservationId`.
- POST /api/v1/reservations/{id}/cancel with that id → **Execute**.

Expected: **200 OK** `status: "Cancelled"` with `cancelledAtUtc` set. The unit is released (Step 8 again shows `availableQuantity: 1`).

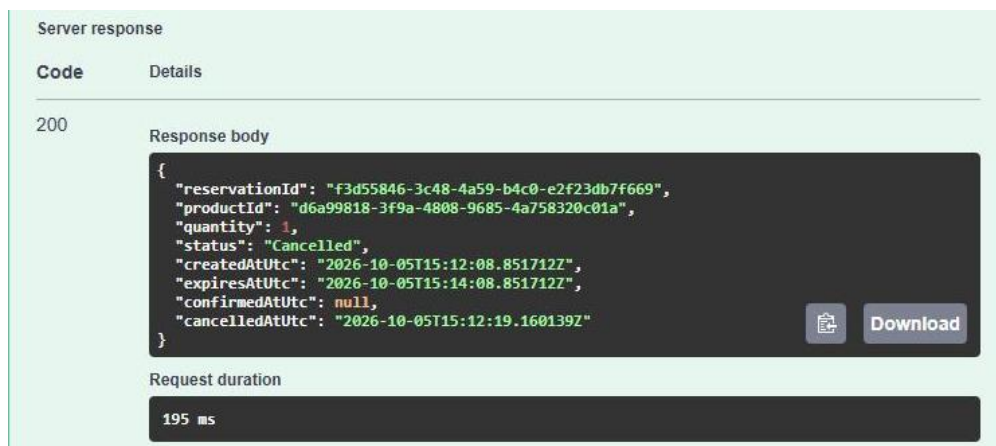


Figure 10: Reservation cancelled and stock released

Step 10. Automatic expiry after 2 minutes

- Reserve again: POST /api/v1/reservations with quantity: 1. Copy the `reservationId` and note `expiresAtUtc`.
- **Wait about 2 minutes 15 seconds** (the background worker sweeps every 15 seconds).
- Run GET /api/v1/products/{id}: `reservedQuantity` is back to 0 and `availableQuantity` to 1.
- Try to confirm the expired reservation: POST /api/v1/reservations/{id}/confirm.

Expected: **409 Conflict** `errorCode: "RESERVATION_EXPIRED"`. A reservation older than 2 minutes can never be bought.

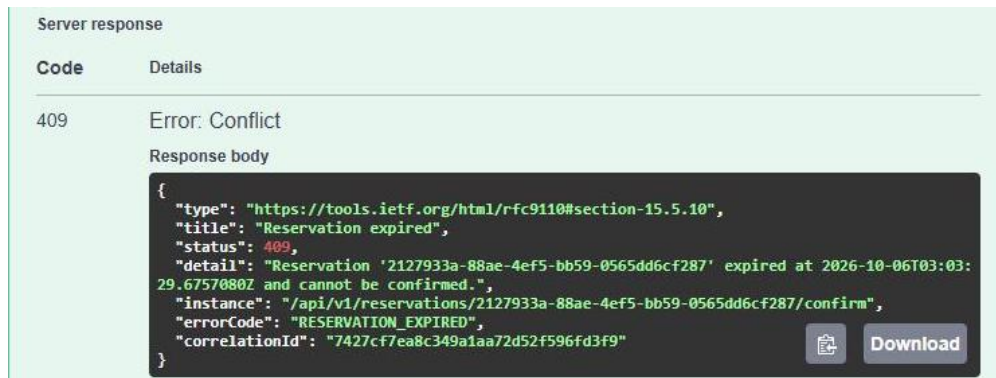


Figure 11: An expired reservation cannot be confirmed

Step 11. Validation error

Endpoint: **POST** `/api/v1/reservations`

```
{
  "productId": "<product id>",
  "quantity": 0
}
```

Expected: **400 Bad Request** `errorCode: "VALIDATION_FAILED"` with a per-field message under `errors.quantity`.

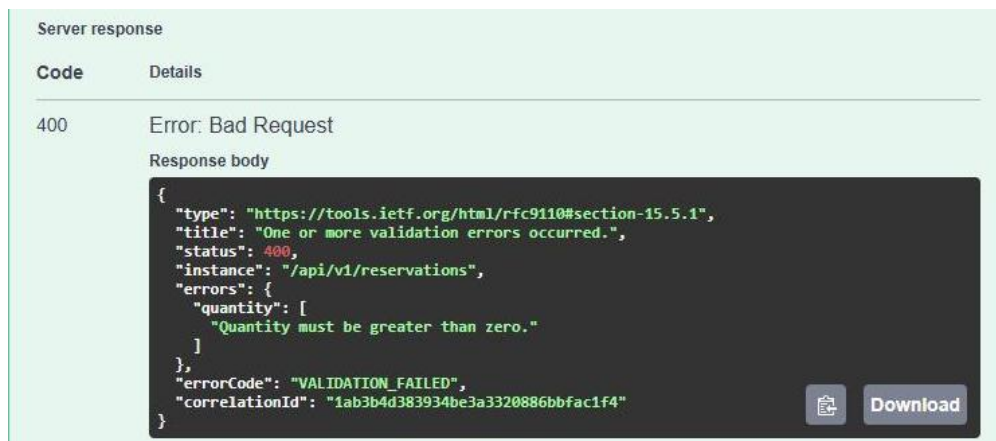


Figure 12: Validation error

Step 12. Call a protected endpoint without a token

- Click **Authorize** → **Logout** → **Close**. The padlocks open again.
- Run **GET** `/api/v1/products/{id}` again.

Expected: **401 Unauthorized** `errorCode: "UNAUTHORIZED"`, detail "A valid bearer token is required."
Authorize again (Step 2) to continue.



Figure 13: Request without a token is rejected

Checklist

Tick each row as you go. Every error has the same JSON shape, with an `errorCode` and a `correlationId`.

#	Action	Expected	Result
1	Login	200, JWT returned	☐
2	Authorize	Padlocks closed	☐
3	Create product (stock 3)	201, available 3	☐
4	Reserve 2	201, Active, expires in 2 minutes	☐
5	Reserve 2 more	409 INSUFFICIENT_STOCK	☐
6	Confirm	200, Confirmed	☐
7	Confirm again	409 INVALID_RESERVATION_STATE	☐
8	Get product	200, reserved 0 / sold 2 / available 1	☐
9	Reserve 1 + cancel	200, Cancelled, stock released	☐
10	Reserve 1, wait 2 min, confirm	409 RESERVATION_EXPIRED, stock released	☐
11	Reserve quantity 0	400 VALIDATION_FAILED	☐
12	No token	401 UNAUTHORIZED	☐

Optional: a normal (non-admin) user who tries to create a product gets **403 FORBIDDEN**. Only the admin is created by default, so this case is checked by the automated tests instead.

4. Run the automated tests

You need the .NET 9 SDK for this. Run the commands from the project folder.

4.1 Quick run (no database needed)

```
dotnet test
```

Expected: **248 tests: 197 passed, 51 skipped, 0 failed** (about 30 seconds). The skipped tests are the ones that need MySQL; 4.2 runs them too.

4.2 Full run, including MySQL and the 500-request test

1. Make sure MySQL is running:

```
docker compose up -d mysql
```

2. Tell the tests how to reach MySQL. Use the `MYSQL_ROOT_PASSWORD` from your `.env`, and its `MYSQL_PORT` if you changed it from 3306.

```
# macOS / Linux / Git Bash
export INVENTORY_TEST_MYSQL=\
"Server=localhost;Port=3306;User=root;Password=<MYSQL_ROOT_PASSWORD>"

# Windows PowerShell
$env:INVENTORY_TEST_MYSQL = `
"Server=localhost;Port=3306;User=root;Password=<MYSQL_ROOT_PASSWORD>"
```

3. Run the tests in the **same** terminal:

```
dotnet test
```

Expected: **248 tests: 248 passed, 0 failed** (about 2 minutes).

Your app data is safe: each test creates its own temporary database and deletes it afterwards.

4.3 Run only the concurrency tests

Handy for running the race tests several times in a row (keep `INVENTORY_TEST_MYSQL` set):

```
dotnet test tests/Inventory.IntegrationTests \
--filter "FullyQualifiedNames~Concurrent|FullyQualifiedNames~Race"
```

4.4 What the concurrency tests prove

Scenario	Result
500 requests for 1 item	1 success, 499 rejected, no overselling
500 requests for 100 items	100 successes, 400 rejected, no overselling
Same, through the full HTTP API with 500 different users	100 × 201, 400 × 409, none rate limited
20 confirms of the same reservation	Exactly 1 succeeds, sold once
20 cancels of the same reservation	Exactly 1 succeeds, stock returned once
Confirm and cancel at the same moment	Exactly one wins each time
8 expiry jobs running at once	Each reservation expired once
Expiry vs confirm, expiry vs cancel	Exactly one wins, stock returned at most once

After every run the test also checks the stock numbers stored in the database.

5. Stop or reset

Command	What it does
<code>docker compose down</code>	Stops the app. Your data is kept for next time.
<code>docker compose down -v</code>	Stops the app and deletes all data . Next start is completely fresh.

6. Troubleshooting

Problem	Fix
<code>docker compose</code> says a value must be set in <code>.env</code>	You skipped step 2 or 3 of section 2.2. Create <code>.env</code> and fill in every value.
Port 3306 or 8080 is already in use	Change <code>MYSQL_PORT</code> or <code>API_PORT</code> in <code>.env</code> , then run <code>docker compose up -d</code> again.
Swagger page does not open	Check <code>SWAGGER_ENABLED=true</code> in <code>.env</code> , then run <code>docker compose up -d</code> again.
Login fails with the correct password	The admin user is only created on the very first start. If you changed the password later, run <code>docker compose down -v</code> and start again.
All MySQL tests are skipped	Set <code>INVENTORY_TEST_MYSQL</code> in the same terminal where you run <code>dotnet test</code> .
Tests time out on a slow machine	Give Docker Desktop more CPU and memory, and close other heavy programs.

7. How it works

7.1 The idea in plain words

Each product stores three numbers: **total** stock, how many are **reserved**, and how many are **sold**. What is still available is simply whatever is left over.

- **Reserve:** move some units from available to reserved.
- **Confirm:** move them from reserved to sold.
- **Cancel or expire:** give them back to available.

A reservation starts as **Active** and can end only once: Confirmed, Cancelled or Expired. A confirmed reservation can never be cancelled, and an expired one can never be confirmed.

7.2 How overselling is prevented

The app never does "read the stock, check it in C#, then save". Between the read and the save, someone else could take the same unit. Instead, **the check and the change happen together, in one database statement**:

Reserving stock

```
UPDATE Products
SET    ReservedQuantity = ReservedQuantity + @quantity
WHERE  Id = @productId
AND    (enough stock is still available);
```

- If the row was updated, the stock was available and is now held. The reservation is saved **in the same transaction**, so both are saved, or neither is.
- If no row was updated, there was not enough stock, and the user gets **409 INSUFFICIENT_STOCK**.

When 500 requests arrive at once, MySQL lines them up on that product and checks each one against the latest numbers. Only as many succeed as there is stock.

Confirm, cancel and expire work the same way: each one only succeeds **if the reservation is still Active**. If two requests race, exactly one wins and the other gets a **409**.

Why not a C# lock? A lock only works inside one running copy of the API. With two copies, each has its own lock and overselling comes back. The database is shared by every copy, so that is where the rule is enforced.

If the database is very busy, MySQL may occasionally abort a transaction. The API retries it up to 3 times; that is safe because each step can only apply once. If all tries fail, nothing was changed and the user gets **503 TEMPORARILY_UNAVAILABLE** and can try again.

7.3 Expiry

- Every reservation expires 2 minutes after it is created.
- A background job checks every 15 seconds, marks overdue reservations as Expired, and puts their stock back.
- Confirming after 2 minutes is always refused with **409 RESERVATION_EXPIRED**, even if the job has not run yet.
- Several copies of the API can run the job at once safely; each reservation is still expired only once.

7.4 Security

- **Login** returns a JWT token, valid for 60 minutes. Send it on other requests as **Authorization: Bearer <token>**.
- **Every endpoint needs a token** except login, health and Swagger.
- Only an **Admin** can create products. Users can confirm or cancel only **their own** reservations (otherwise **403 FORBIDDEN**).
- Passwords are stored as strong one-way hashes, never as plain text. Secrets come from **.env**, never from the code, and are never logged.

7.5 Errors, logging and rate limiting

- Every error has the same JSON shape with a clear **errorCode** (for example **INSUFFICIENT_STOCK**) and a **correlationId**.
- Every request is logged as structured JSON with that correlation ID, so a reported error can be found in the logs.
- Each user has a request limit (for example 10 login attempts per minute). Going over it returns **429**.

7.6 Project structure

Project	What lives there
Inventory.Domain	The business objects (Product , Reservation , User) and their rules. No database or web code.
Inventory.Application	The use cases: create, confirm, cancel and expire reservations; create products; log in.
Inventory.Infrastructure	Talking to MySQL (EF Core), password hashing, creating JWT tokens.
Inventory.Api	Controllers, login checks, error responses, logging, rate limiting, Swagger, the expiry job.
Inventory.UnitTests / IntegrationTests	84 unit tests and 164 integration tests.

7.7 API endpoints

Endpoint	Who can call it	What it does
POST /api/v1/auth/login	anyone	Log in and get a token
POST /api/v1/products	Admin	Create a product
GET /api/v1/products/{id}	logged-in user	See a product and its stock
POST /api/v1/reservations	logged-in user	Reserve stock for 2 minutes
POST /api/v1/reservations/{id}/confirm	owner or Admin	Buy the reserved stock
POST /api/v1/reservations/{id}/cancel	owner or Admin	Give the stock back
GET /health · /health/ready	anyone	Is the API (and database) up?

8. Files to read first

The important logic is in a few files:

Topic	Files (in reading order)
Create Reservation	ReservationsController.cs → ReservationService.cs → Reservation.cs
Overselling protection	ReservationRepository.cs , TransientMySqlRetry.cs
Confirm / Cancel	Reservation.cs , ReservationService.cs , ReservationRepository.cs
Expiry	ReservationExpiryWorker.cs → ReservationExpiryService.cs
Login / JWT	AuthService.cs , JwtTokenGenerator.cs , AuthenticationSetup.cs
The 500-request test	tests/.../Reservations/ReservationConcurrencyTests.cs

9. Settings reference (.env)

Setting	What it is	Required
MYSQL_ROOT_PASSWORD	MySQL admin password (also used by the tests)	yes
MYSQL_PASSWORD	Password the app uses to connect to MySQL	yes
JWT_SIGNING_KEY	Secret used to sign login tokens, at least 32 characters	yes
DEMO_ADMIN_USERNAME / DEMO_ADMIN_PASSWORD	The admin user created on first start	no
MYSQL_PORT / API_PORT	Ports on your computer (default 3306 / 8080)	no
MYSQL_DATABASE / MYSQL_USER	Database and user names (default inventory / inventory)	no
SWAGGER_ENABLED	Show the Swagger page (<code>true</code> in the example file)	no

`.env` holds your secrets. It is never committed to git and never copied into the Docker image.

10. Assumptions and future improvements

Assumptions

- A reservation is for one product and a quantity of at least 1. Ids are GUIDs.
- Users are created from configuration; there is no sign-up endpoint.
- An expired reservation may wait up to 15 seconds for the background job, but it can never be confirmed in that time.
- In production, HTTPS is handled by a reverse proxy in front of the API.

Future improvements

- **Very popular products:** all requests for one product wait in line for it. Stock stays correct, but in a big flash sale requests get slower. Fixes: fail fast after a short wait, or limit requests per product.
- **Idempotency keys**, so a client that retries a request does not create two reservations.
- **Shared rate limits** across all API copies (for example, with Redis).
- Let expiry jobs on different API copies split the work instead of overlapping.

The repository README.md has the same information in a shorter form.